



Faculty of Computer Applications

**MCA Sem-1
(AY-2026-27)**

Unit-2 Assignment

**Sub Code: 05MC0108
(DBMS)**

Enrollment No :	92600584037
Full Name :	Darshit Mehta

Subject Faculty Name: Prof. Amit Gohel

Index

<u>No.</u>	<u>Unit-2</u>
1	Create a procedure that accepts Department Number and displays: ❑ Total Employees ❑ Average Salary ❑ Maximum Salary Use OUT parameters.
2	Create a function that returns the annual salary of an employee (salary × 12). Call the function inside a SELECT statement.
3	Create a procedure to update the salary of an employee using the employee ID. Handle the exception if the employee ID does not exist.
4	Create a package named PKG_EMPLOYEE containing: ❑ Function to calculate the annual salary of an employee. ❑ Procedure to show employee information using the employee ID. Call each package object separately.
5	Create a function that returns employee experience category based on joining date. Categories: ❑ Less than 2 Years → Fresher ❑ 2–5 Years → Experienced ❑ More than 5 Years → Senior Employee

1. Create a procedure that accepts Department Number and displays:

? Total Employees

? Average Salary

? Maximum Salary

Use OUT parameters.

-- 1. Procedure Create

CREATE OR REPLACE PROCEDURE DISPLAY(

DPNO IN EMP.DEPTNO%TYPE,

TOTAL_EMP OUT NUMBER,

AVG_SALARY OUT NUMBER,

MAX_SALARY OUT NUMBER

) AS

BEGIN

SELECT COUNT(*),

AVG(BASICSAL),

MAX(BASICSAL)

INTO TOTAL_EMP,

AVG_SALARY,

MAX_SALARY

FROM EMP

WHERE DEPTNO = DPNO;

EXCEPTION

WHEN OTHERS THEN

DBMS_OUTPUT.PUT_LINE('NOT FOUND');

END;

/

-- 2. Create OUT Variables

```
VAR TOTAL_EMP NUMBER;
```

```
VAR AVG_SALARY NUMBER;
```

```
VAR MAX_SALARY NUMBER;
```

-- 3. Execute Procedure

```
EXECUTE DISPLAY(10, :TOTAL_EMP, :AVG_SALARY, :MAX_SALARY);
```

-- 4. Display Output

```
PRINT TOTAL_EMP;
```

```
PRINT AVG_SALARY;
```

```
PRINT MAX_SALARY;
```

Output : -

```
SQL> PRINT TOTAL_EMP;
TOTAL_EMP
-----
          1

SQL> PRINT AVG_SALARY;
AVG_SALARY
-----
       50000

SQL> PRINT MAX_SALARY;
MAX_SALARY
-----
       50000
```

2. Create a function that returns the annual salary of an employee (salary × 12). Call the function inside a SELECT statement.

```
CREATE TABLE EMPLOYEE (  
    EMP_ID NUMBER,  
    EMP_NAME VARCHAR2(50),  
    SALARY NUMBER  
);
```

```
SELECT EMP_ID,  
    EMP_NAME,  
    SALARY,  
    ANNUAL_SALARY(SALARY) AS ANNUAL_SALARY  
FROM EMPLOYEE;
```

```
CREATE OR REPLACE FUNCTION ANNUAL_SALARY(P_SALARY NUMBER)  
RETURN NUMBER  
IS  
BEGIN  
    RETURN P_SALARY * 12;  
END;  
  
/  
  
SELECT ANNUAL_SALARY(25000) AS ANNUAL_SALARY  
FROM DUAL;
```

Output :-

```
SQL> SELECT ANNUAL_SALARY(25000) AS ANNUAL_SALARY  
2 FROM DUAL;
```

```
ANNUAL_SALARY  
-----  
300000
```

3. Create a procedure to update the salary of an employee using the employee ID. Handle the exception if the employee ID does not exist.

-- Create Table

```
CREATE TABLE EMPLOYEE (  
    EMP_ID NUMBER,  
    EMP_NAME VARCHAR2(50),  
    SALARY NUMBER  
);
```

-- Insert Records

```
INSERT INTO EMPLOYEE VALUES (101, 'DARSHIT', 25000);  
INSERT INTO EMPLOYEE VALUES (102, 'HIT', 30000);
```

```
COMMIT;
```

-- Create Procedure

```
CREATE OR REPLACE PROCEDURE UPDATE_SALARY(  
    P_EMP_ID IN NUMBER,  
    P_SALARY IN NUMBER  
)  
IS  
BEGIN
```

```
    UPDATE EMPLOYEE  
    SET SALARY = P_SALARY  
    WHERE EMP_ID = P_EMP_ID;
```

```
    IF SQL%ROWCOUNT = 0 THEN  
        RAISE NO_DATA_FOUND;  
    END IF;
```

```
    DBMS_OUTPUT.PUT_LINE('SALARY UPDATED SUCCESSFULLY.');
```

```
EXCEPTION  
    WHEN NO_DATA_FOUND THEN
```

```

        DBMS_OUTPUT.PUT_LINE('EMPLOYEE ID DOES NOT EXIST.');
```

END;
/
-- Enable Output

```
SET SERVEROUTPUT ON;
```

-- Execute Procedure

```
BEGIN
    UPDATE_SALARY(101, 50000);
END;
```

/

-- Display Employee Records

```
SELECT * FROM EMPLOYEE;
```

Output :-

```

SQL> SET SERVEROUTPUT ON;
SQL> BEGIN
  2     UPDATE_SALARY(101, 50000);
  3 END;
  4 /
SALARY UPDATED SUCCESSFULLY.

PL/SQL procedure successfully completed.

SQL> SELECT * FROM EMPLOYEE;
```

EMP_ID	EMP_NAME	SALARY
101	DARSHIT	50000
102	HIT	30000

4. Create a package named PKG_EMPLOYEE containing:

❑ **Function to calculate the annual salary of an employee.**

❑ **Procedure to show employee information using the employee ID. Call each package object separately.**

SET SERVEROUTPUT ON;

CREATE OR REPLACE PACKAGE PKG_EMPLOYEE
AS

FUNCTION ANNUAL_SALARY(
P_EMP_ID IN EMP.EID%TYPE
)
RETURN NUMBER;

PROCEDURE SHOW_EMPLOYEE(
P_EMP_ID IN EMP.EID%TYPE
);

END PKG_EMPLOYEE;
/

CREATE OR REPLACE PACKAGE BODY PKG_EMPLOYEE
AS

FUNCTION ANNUAL_SALARY(
P_EMP_ID IN EMP.EID%TYPE
)
RETURN NUMBER
IS
V_SALARY EMP.BASICSAL%TYPE;
BEGIN

SELECT BASICSAL
INTO V_SALARY
FROM EMP
WHERE EID = P_EMP_ID;

RETURN V_SALARY * 12;

EXCEPTION

```
    WHEN NO_DATA_FOUND THEN
        DBMS_OUTPUT.PUT_LINE('EMPLOYEE ID NOT FOUND');
        RETURN 0;
END ANNUAL_SALARY;
```

```
PROCEDURE SHOW_EMPLOYEE(
    P_EMP_ID IN EMP.EID%TYPE
)
IS
    V_NAME EMP.ENAME%TYPE;
    V_SAL EMP.BASICSAL%TYPE;
    V_DNO EMP.DEPTNO%TYPE;
BEGIN
```

```
    SELECT ENAME, BASICSAL, DEPTNO
    INTO V_NAME, V_SAL, V_DNO
    FROM EMP
    WHERE EID = P_EMP_ID;
```

```
    DBMS_OUTPUT.PUT_LINE('EMPLOYEE ID : ' || P_EMP_ID);
    DBMS_OUTPUT.PUT_LINE('EMPLOYEE NAME : ' || V_NAME);
    DBMS_OUTPUT.PUT_LINE('SALARY      : ' || V_SAL);
    DBMS_OUTPUT.PUT_LINE('DEPARTMENT NO : ' || V_DNO);
```

```
EXCEPTION
    WHEN NO_DATA_FOUND THEN
        DBMS_OUTPUT.PUT_LINE('EMPLOYEE ID NOT FOUND');
END SHOW_EMPLOYEE;
```

```
END PKG_EMPLOYEE;
/
```

```
SELECT PKG_EMPLOYEE.ANNUAL_SALARY(102) AS ANNUAL_SALARY
FROM DUAL;
```

```
BEGIN
    PKG_EMPLOYEE.SHOW_EMPLOYEE(102);
END;
/
```

Output :-

```
SQL> SELECT PKG_EMPLOYEE.ANNUAL_SALARY(102) AS ANNUAL_SALARY  
2 FROM DUAL;
```

```
ANNUAL_SALARY  
-----  
          900000
```

```
SQL> BEGIN  
2     PKG_EMPLOYEE.SHOW_EMPLOYEE(102);  
3 END;  
4 /
```

```
EMPLOYEE ID   : 102  
EMPLOYEE NAME : Hit  
SALARY        : 75000  
DEPARTMENT NO : 20
```

```
PL/SQL procedure successfully completed.
```

5. Create a function that returns employee experience category based on joining date. Categories:

❑ **Less than 2 Years → Fresher**

❑ **2–5 Years → Experienced**

❑ **More than 5 Years → Senior Employee.**

```
SET SERVEROUTPUT ON;
```

```
ALTER TABLE EMP  
ADD JOINING_DATE DATE;
```

```
UPDATE EMP  
SET JOINING_DATE = DATE '2025-01-01'  
WHERE EID = 101;
```

```
UPDATE EMP  
SET JOINING_DATE = DATE '2023-01-01'  
WHERE EID = 102;
```

```
UPDATE EMP  
SET JOINING_DATE = DATE '2018-01-01'  
WHERE EID = 103;
```

```
COMMIT;
```

```
CREATE OR REPLACE FUNCTION EMP_EXPERIENCE(  
    P_EMP_ID IN EMP.EID%TYPE  
)  
RETURN VARCHAR2  
IS  
    V_JOINING_DATE EMP.JOINING_DATE%TYPE;  
    V_YEARS NUMBER;  
BEGIN
```

```
    SELECT JOINING_DATE  
    INTO V_JOINING_DATE  
    FROM EMP  
    WHERE EID = P_EMP_ID;
```

```
    V_YEARS := MONTHS_BETWEEN(SYSDATE, V_JOINING_DATE) / 12;
```

```
IF V_YEARS < 2 THEN
```

```
    RETURN 'FRESHER';
```

```
ELSIF V_YEARS <= 5 THEN
```

```
    RETURN 'EXPERIENCED';
```

```
ELSE
```

```
    RETURN 'SENIOR EMPLOYEE';
```

```
END IF;
```

```
EXCEPTION
```

```
    WHEN NO_DATA_FOUND THEN
```

```
        RETURN 'EMPLOYEE ID NOT FOUND';
```

```
END;
```

```
/
```

```
SELECT EID,
```

```
    ENAME,
```

```
    JOINING_DATE,
```

```
    EMP_EXPERIENCE(EID) AS EXPERIENCE_CATEGORY
```

```
FROM EMP;
```

Output :-

```
SQL> SELECT EID,  
2         ENAME,  
3         JOINING_DATE,  
4         EMP_EXPERIENCE(EID) AS EXPERIENCE_CATEGORY  
5 FROM EMP;
```

EID	ENAME	JOINING_D
-----	-------	-----------

EXPERIENCE_CATEGORY

101 Ronak	01-JAN-25
FRESHER	

102 Hit	01-JAN-23
EXPERIENCED	

103 Pranav	01-JAN-18
SENIOR EMPLOYEE	